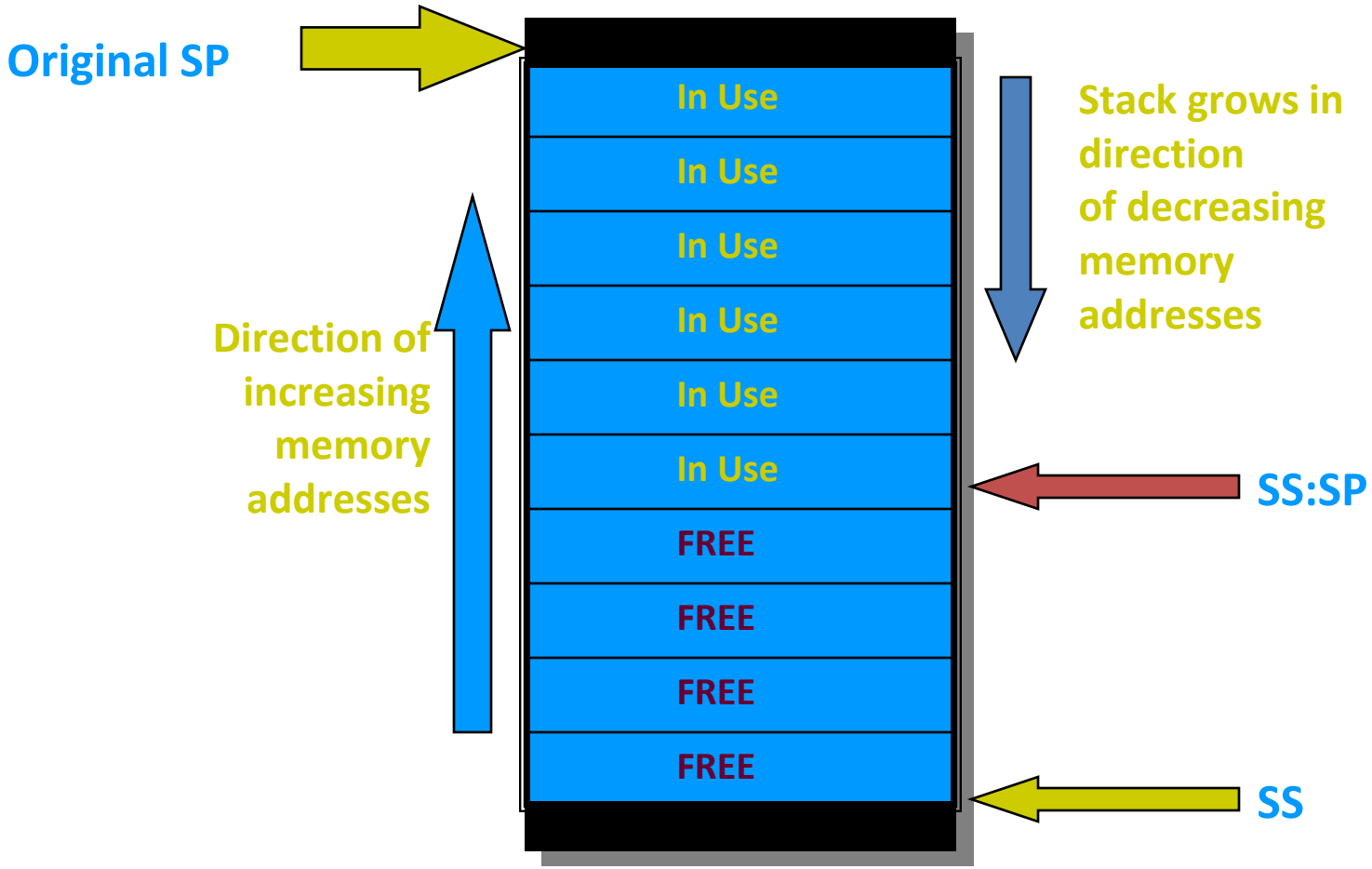
The Stack and Procedures

The Stack

- The stack segment of a program is used for temporary storage of data and addresses
- A stack is a one-dimensional data structure
- Items are added to and removed from one end of the structure using a "Last In - First Out" technique (LIFO)
- The top of the stack is the last addition to the stack
- The SS (Stack Segment Register) contains the segment number of the stack segment
- The complete segment:offset address to access the stack is SS:SP

- Initially before any data or addresses have been placed on the stack, the SP contains the offset address of the memory location immediately following the stack segment

Stack Implementation in Memory



PUSH Instruction

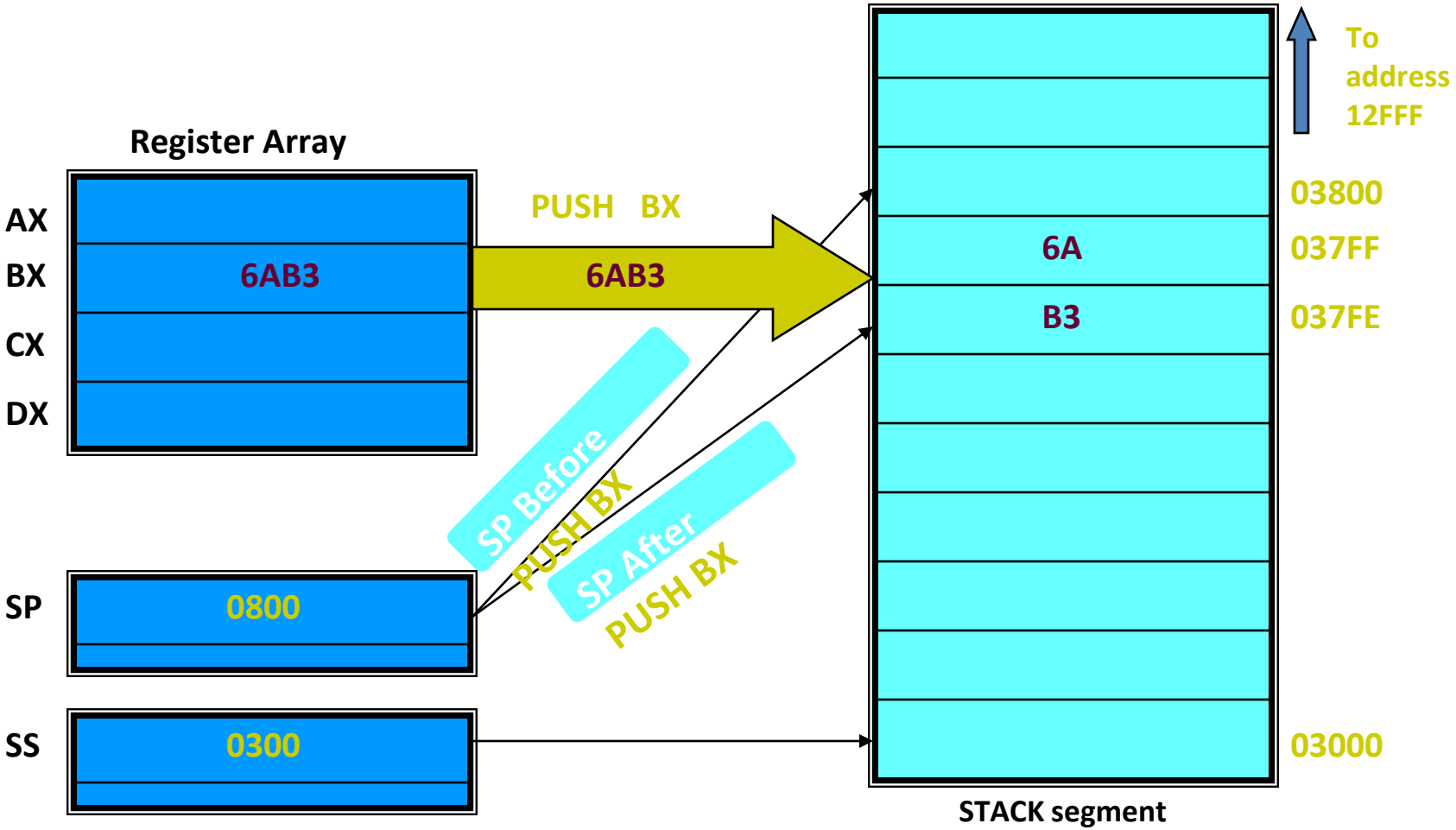
- PUSH instruction adds a new word to the stack
- SYNTAX: PUSH source

  where source is a 16-bit register or memory word
- PUSH instruction causes the stack pointer (SP) to be decreased by 2. Then a copy of the value in the source field is placed in the address specified by SS:SP.
- Because each PUSH decreases the SP, the stack is filled a word at a time backwards from the last available word in the stack toward the beginning of the stack.

Ex: PUSH AX

- PUSH DS
- PUSH WORD PTR [SI]

PUSH Instruction Example



POP Instruction

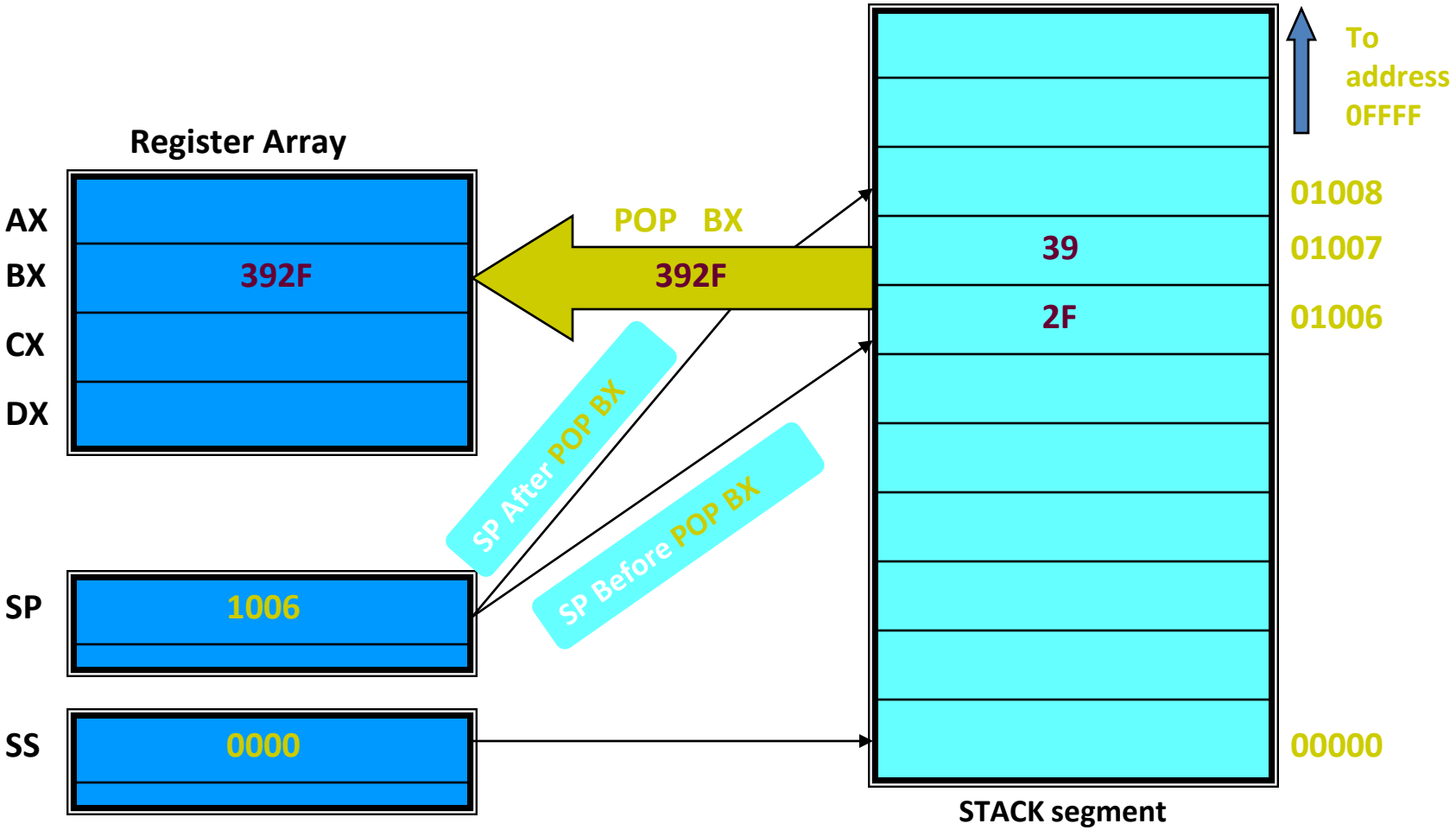
- POP instruction removes the last word placed on the stack
- SYNTAX: POP destination
  - where source is a 16-bit register or memory word
- POP instruction causes the contents of SS:SP to be moved to the destination field
- It increases the stack pointer (SP) by 2
- ***Restrictions:***
  1. PUSH and POP work only with words
  2. Byte and immediate data operands are illegal

EX: POP AX

POP CS

POP WORD PTR [DI]

POP Instruction Example



FLAGS Register and Stack

- PUSHF
  - pushes (copies) the contents of the FLAGS register onto the stack. It has no operands
- POPF
  - pops (copies) the contents of the top word in the stack to the FLAGS register. It has no operands

- ***NOTES:***
  - *PUSH, POP, and PUSHF do not affect the flags !!*
  - *POPF could theoretically change all the flags because it resets the FLAGS REGISTER to some original value that you have previously saved with the PUSHF instruction*

Procedure and Macro

- A procedure is group of instructions that usually performs one task. It is a reusable section of a software program which is stored in memory once but can be used as often as necessary.
- A procedure can be of two types.
  1) Near Procedure
  2) Far Procedure

- **Near Procedure:** A procedure is known as NEAR procedure if is written(defined) in the same code segment which is calling that procedure. Only Instruction Pointer(IP register) contents will be changed in NEAR procedure.
- **FAR procedure :** A procedure is known as FAR procedure if it is written (defined) in the different code segment than the calling segment. In this case both Instruction Pointer(IP) and the Code Segment(CS) register content will be changed.

Directives used for procedure

- **PROC directive:** The PROC directive is used to identify the start of a procedure. The PROC directive follows a name given to the procedure. After that the term FAR and NEAR is used to specify the type of the procedure.
- **ENDP Directive:** This directive is used along with the name of the procedure to indicate the end of a procedure to the assembler. The PROC and ENDP directive are used to bracket a procedure.

CALL instruction and RET instruction

- **CALL instruction** : The CALL instruction is used to transfer execution to a procedure. It performs two operation. When it executes, first it stores the address of instruction after the CALL instruction on the stack. Second it changes the content of IP register in case of Near call and changes the content of IP register and cs register in case of FAR call.
- There are two types of calls.
    1)Near Call or Intra segment call.
    2) Far call or Inter Segment call

Operation for Near Call

- When 8086 executes a near CALL instruction, it decrements the stack pointer by 2 and copies the IP register contents on to the stack. Then it copies address of first instruction of called procedure.

  SP <- SP-2

  IP -> stores onto stack (SS:SP)

  IP <- starting address of a procedure.

- Direct Call: CALL PROC1

  CALL PRT_MES

- Indirect Call: CALL AX

  CALL word ptr[SI]

Operation of FAR CALL

When 8086 executes a far call:

it decrements the stack pointer by 2 and copies the contents of CS register to the stack.

It the decrements the stack pointer by 2 again and copies the content of IP register to the stack.

Finally it loads cs register with base address of segment having procedure and IP with address of first instruction in procedure.

SP <- sp-2, CS contents ->stored on stack

SP <- sp-2, IP contents -> stored on stack

CS <- Base address of segment having procedure

IP <- address of first instruction in procedure.

Direct and Indirect CALL

- Direct intersegment Call:

  CALL FAR PTR proc_name
  - SP<-sp-2
  - (SS:SP)<-(CS)
  - SP<-SP-2
  - (SS:SP)<-(IP)
  - (CS)<-segment base of the procedure proc_name
  - (IP) <-offset of the procedure proc_name
- Indirect CALL:

  CALL DWORD PTR [SI]
  (IP)<-(DS:SI)
  (CS)<-(DS:SI+2)

RET instruction

- The RET instruction will return execution from a procedure to the next instruction after call in the main program. At the end of every procedure RET instruction must be executed.
- **Operation for Near Procedure :** For NEAR procedure ,the return is done by replacing the IP register with a address popped off from stack and then SP will be incremented by 2.

  IP <- Address from top of stack

  SP <- SP+2
- **Operation for FAR procedure :** IP register is replaced by address popped off from top of stack, then SP will be incremented by 2.The CS register is replaced with a address popped off from top of stack.Again SP will be incremented by 2.

  IP <- Address from top of stack

  SP <- SP+2

  CS <- Address from top of stack

  SP <- SP+2

Difference between FAR CALL and NEAR CALL

Near Call	**Far Call**
A near call refers a procedure which is in the same code segment	A Far call refers a procedure which is in different code segment
It is also called Intra-segment call.	It is also called Inter-segment call
A Near Call replaces the old IP with new IP	A FAR replaces CS & IP with new CS & IP.
It uses keyword near for calling procedure	It uses keyword far for calling procedure.
Less stack locations are required	More stack locations are required.

Parameter Passing in Procedures

- We often want a procedure to process some data or address variable from the main program. For processing, it is necessary to pass these address variables or data, usually referred as Parameter Passing Techniques. There are four ways to pass parameters to and from the procedure
  1. Using registers
  2. Using general memory
  3. Using pointers
  4. Using stack

Parameter passing using registers

- The general purpose registers are used to pass the parameters to the procedure and return the value from the procedure.
- For Ex:

```
MOV AX, num
CALL bin_cnt
MOV count,AL
```

Disadv: Number of registers limits the number of parameters that can be passed.

Parameter passing using general memory

In this the procedure directly access the parameters by name from the procedure.

For Ex:

```
CALL bin_cnt
Bin_cnt proc
  MOV AX,num
  …
  Mov count,AL
```

Disadv: It always look to the memory location named num,count to get data and to store the result and hence we cannot easily use this procedure to count binary number in some other memory location

Parameter passing using pointers

- This overcomes the disadvantage of earlier two methods.
- In this registers are used to pass the procedure pointers of the desired data.

For Ex:

```
MOV SI, offset  num
MOV DI, offset count
CALL bin_cnt

bin_cnt proc
MOV AX,[SI]
…
MOV [DI],AL
```

Parameter passing using stack

- Most of the programming language use this method of passing the parameters.
- Before the procedure call parameters are pushed onto stack, which are then accessed by the procedure and the procedure also stores the return value on the stack which caller reads from the stack.
- Ex: MOV AX,num

```
PUSH AX
CALL bin_cnt
POP AX
MOV count,AL
```

But while passing the parameters using stack care should be taken for pointing at correct address with the help of BP register.

```
MOV AX, num
PUSH AX
CALL bin_cnt
POP AX
MOV count,AL
INT 03

 bin__cnt PROC NEAR
      PUSHF
      PUSH AX
       PUSH BX
       PUSH CX
      PUSH BP
      MOV AX,[BP+12]

      …
```

```
MOV [BP+12],AX
POP BP
POP CX
POP BX
POP AX
POPF
RET
Bin_cnt ENDP
Code ends
end
```

Reentrant and Recursive procedures

- Reentrant procedures: The procedure which can be interrupted, used and “reentered” without losing or writing over anything.
- Recursive procedure: It is the procedure which call itself.

Reentrant procedure

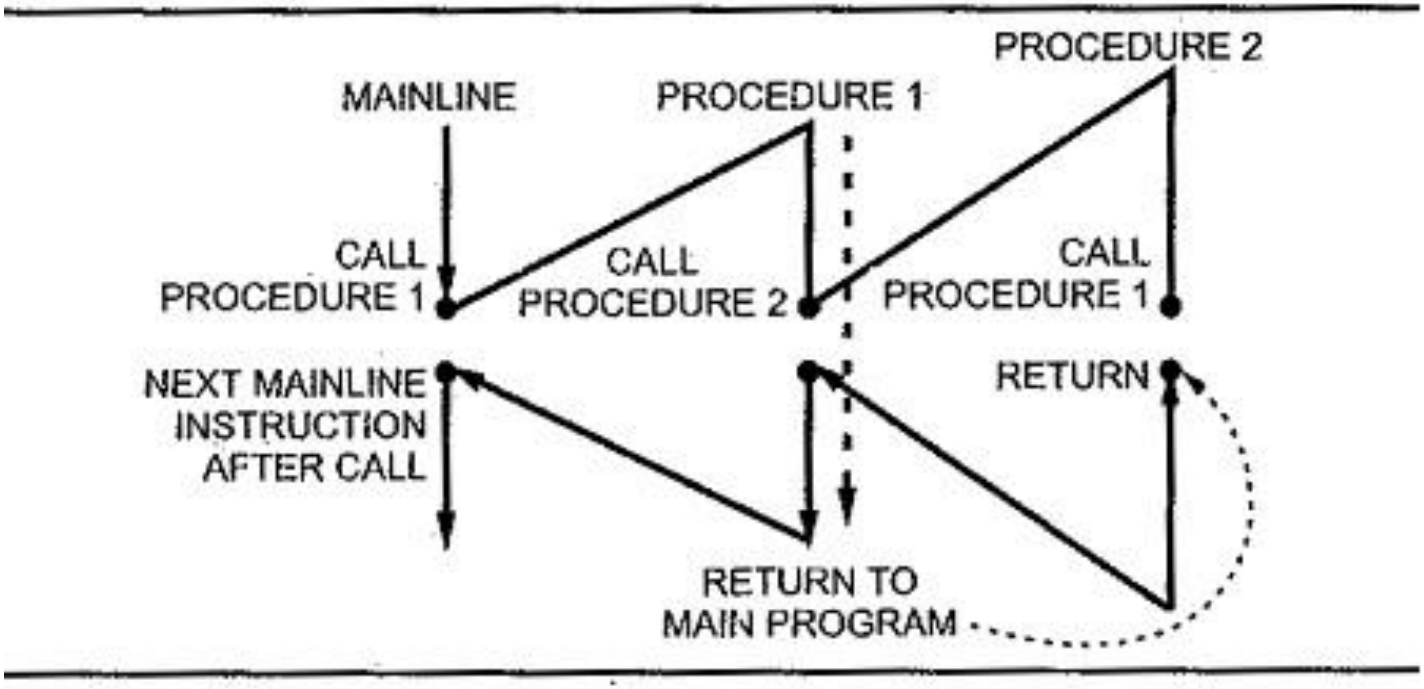


Fig. 8.3 Flow of program execution for reentrant procedure

Recursive procedure

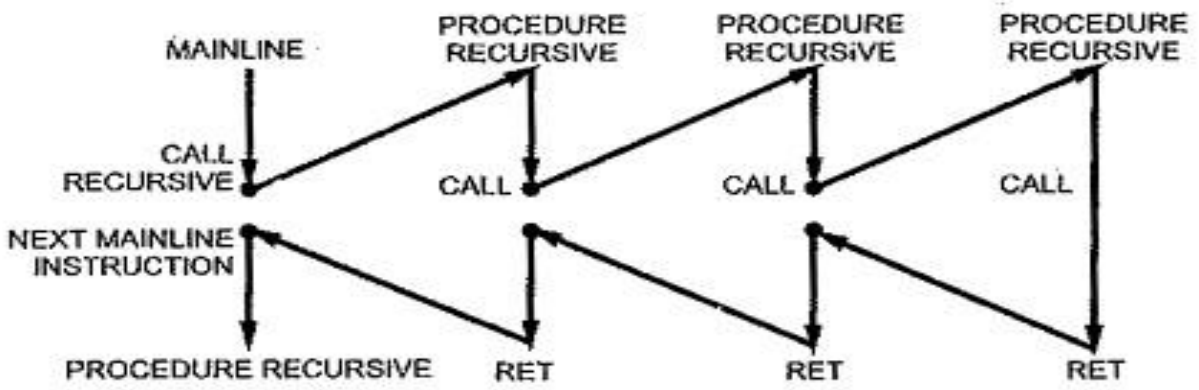


Fig. 8.4 Flow diagram and pseudo-code for recursive procedure

Assembler Directives used in FAR procedures

- Segment directive uses two type specifiers which are optional.
- Seg_name Segment [word][public]
- The word indicates that the segment should be located in memory at next available even address.
- Public indicates that this segment combined with the other segment in the other assembly module having the same name.

PUBLIC Directive

- The Public directive is used to publicly declare the entries like variables,procedures.
- It informs the assembler that the entries specified are accessible from the other assembly module.
- For Ex: PUBLIC name1,name2
  - PUBLIC num
  - PUBLIC bin_cnt

EXTRN Directive

- The EXTRN directive informs the assembler that the specified entities used in this module are defined in another assembly module.
- EXTRN name1:type, name2:type...nameN:type
  - EXTRN num:word, bin_cnt:far

Label Directive

- The label directive is used to give a name to the current value in the location counter.

Ex:
```
Stack_seg SEGMENT STACK
        DW 100 dup(0)
        STACK_TOP LABEL WORD
    Stack_seg ENDS
```

Location Counter

- Purposes of instruction location counter in an assembler are as follows:
  - The location counter checks whether the next available memory location can be assigned within the present (current) segment, such as code, or data. In other words, it contains the offset of the instruction or the data being assembled.
  - The $ sign gives the current value of the location counter.
  - It is used to locate the next statement with respect to base.
  - By default, the location counter starts from zero and increment by one when each byte of memory is allocated. But the location counter may be changed using the ORG statement.

Ex: str1 DB “hello”

len1 DB $-str1

ORG Directive

- To start the location counter at a particular offset address org assembler directive may be used.
- When assembling a three byte instruction, the location counter will increment by three bytes to accommodate the memory needed for the instruction. Separate location counters are used for different sections of the program. Typically one section is used for code and another for data. Actual locations are fixed during the linking process.
- The following statement forces the location counter to start at offset address 10 hexadecimal.

```
org $10
```

MACRO

- A macro is a named block of assembly language statements.
- Once defined, it can be invoked (called) as many times in a program as we wish.
- When a macro is invoked, a copy of its code is inserted directly into the program at the location where it was invoked.
- Defined directly at the beginning of a source program, or they are placed in a separate file and copied into a program by an INCLUDE directive.

MACRO(Contd)

- Macros are expanded during the assembler's preprocessing step. In this step, the preprocessor reads a macro definition and scans the remaining source code in the program.
- At every point where the macro is called, the assembler inserts a copy of the macro's source code into the program.
- A macro definition must be found by the assembler before trying to assemble any calls of the macro.
- If a program defines a macro but never calls it, the macro code does not appear in the compiled program.

Directives used in MACRO

- MACRO: Indicates start of the Macro and preceded by the name of the macro

  syntax: macroname MACRO parameter-1, parameter-2...
- ENDM: It denotes the end of the macro.
- Ex:

```
EXCH MACRO
    MOV CX,BX
    MOV AX,BX
    MOV BX,CX
ENDM
```

```
sum Macro n1,n2
    MOV AL,n1
    MOV bl,n2
    ADD AL,BL
ENDM
```

- If we want to call above macro just type the name of macro.

Difference Between Procedure and Macro

Procedure	Macro
1. Accessed by CALL and RET instruction during program execution.	1. Accessed during assembly with name given to macro when defined.
2. Machine code for instructions is put only once in the memory.	2. Machine code is generated for instructions each time when macro is called.
3. With procedures less memory is required.	3. With macros more memory is required.
4. Parameters can be passed in registers, memory locations, or stack.	4. Parameters passed as part of statement which calls macro.

INCLUDE Directive

- It is used to include the header files in the program.
- It copies the file specified in the directive in the program.
- The name of the file is specified with full path specification or it is assumed to be in the same directory.
- Ex: INCLUDE macro.h

  INCLUDE d:\xyz\macro.h

GROUP directive

- It can be used to tell the assembler to group the logical segments named after the directive into one logical group. This allows the contents of all the segments to be accessed from the same group.
- Example: SMALL-SYSTEM GROUP CODE, DATA, STACK-SEG.
- ASSUME CS: SMALL-SYSTEM,DS: SMALL-SYSTEM,SS: SMALL-SYSTEM

- **EQU Directive**: The EQU directive is used to give name to some value or symbol. Each time the assembler finds the given names in the program, it will replace the name with the value or a symbol.

  Ex: MAX EQU 10

  MOV CX,MAX

- **TYPE Directive**: TYPE operator instructs the assembler to determine the type of a variable and determines the number of bytes specified to that variable.

  Byte type variable – assembler will give a value 1

  Word type variable – assembler will give a value 2

  Double word type variable – assembler will give a value 4

  Example: ADD SI,TYPE array (SI=SI+2, if array is of type word)

LENGTH and SIZE directive

- The length directive is used to find the length of the data item in terms of elements,(i.e number of bytes for DB, no. of words for DW)
- The size directive always give the size of the data items in terms of bytes.

Data Segment block db 25 dup(0) array dw 50 dup(0) num1 dw 1,2,3,4 Data ends	MOV AX, Length block (AX<-25) MOV AX, size block (AX<-25) MOV AX, Length array (AX<-50) MOV AX, size array (AX<-100) MOV AX, Length num1 (AX<-1) MOV AX, size num1 (AX<-2)


Thank You